Звіт з практичної роботи № 6

Прямі методи розв'язання СЛАР, розріджені матриці та ефект впорядкування
Варіант 3

1. Фізична система та генератор матриці (варіант 3)

Стаціонарна теплопровідність у прямокутній пластині: на сітці m×m внутрішніх вузлів для кожного вузла записується баланс тепла з чотирма сусідами (5-точковий шаблон), на межі температура фіксована (T = 0, умови Діріхле), права частина b = 1 (рівномірне джерело). Коефіцієнт теплопровідності k = 1 у лівій половині пластини й k = c у правій, контраст c = 1…1000; провідність ребра між вузлами - гармонічне середнє k сусідів.

Властивості матриці A (n = m^2): симетрична (провідність ребра однакова в обидва боки); додатно визначена (діагональ строго домінує, оскільки сума за межею дає додатний внесок, а граф зв'язний), тобто SPD; розріджена: ≤ 5 ненульових у рядку, nnz ≈ 5n; стрічкова з напівшириною m в природному порядку вузлів. Тому правильний метод - розклад Холецького (вдвічі дешевший за LU і без півотингу), а LU з частковим півотингом реалізовано для порівняння й для загального випадку.

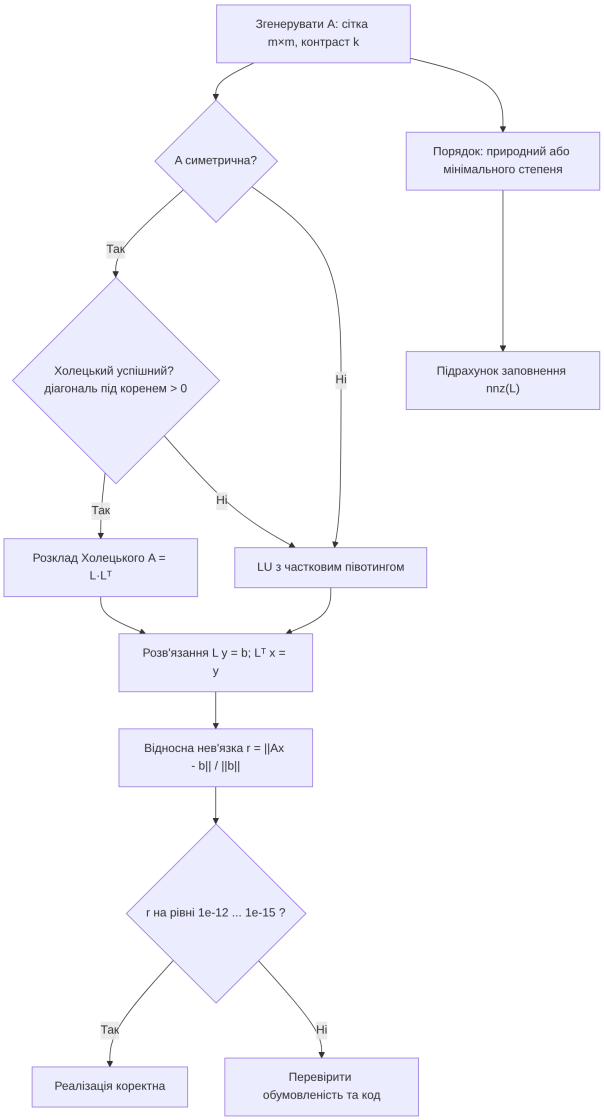


Рис. 1. Вибір методу та перевірка коректності (Mermaid)

2. Реалізовані методи та очікувана складність (для n = m^2)

Метод	Час	Пам'ять	Примітка
LU з частковим півотингом (щільний)	≈ $2n^3/3$ = $O(m^6)$	n^2 = $O(m^4)$	Для довільної матриці; стійкий завдяки півотингу

Метод	Час	Пам'ять	Примітка
Холецький $A = L \cdot L^T$ (щільний)	$\approx n^3/3 = O(m^6)$	$n^2 = O(m^4)$	Лише SPD; приблизно вдвічі дешевший за LU
Холецький, стрічковий (напівширина m)	$O(n \cdot m^2) = O(m^4)$	$n \cdot (m+1) = O(m^3)$	Використовує стрічкову структуру; це «бібліотечний» рівень цієї роботи

3. Коректність: відносна нев'язка r = ‖Ax - b‖ / ‖b‖

Нев'язка обчислюється через множення в форматі CSR для кожного методу. Вимога - рівень машинної похибки добре обумовлених задач (10^{-12}...10^{-15}). Сітка 20×20 (n = 400), різні контрасти:

Контраст c	LU (півотинг)	Холецький (щільний)	Холецький (стрічковий)
1	1.72e-14	1.62e-14	1.62e-14
10	2.27e-14	2.22e-14	2.22e-14
100	2.38e-14	2.16e-14	2.16e-14
1000	1.77e-14	1.97e-14	1.97e-14

Нев'язки близько 10^{-14} для всіх контрастів до 1000 - методи реалізовано правильно; це не лише «програма не впала». Холецький не зупинився на «від'ємному корені», що додатково підтверджує додатну визначеність. Зі збільшенням n нев'язка стрічкового методу зростає (від $\approx 5 \cdot 10^{-15}$ при n = 100 до $\approx 3 \cdot 10^{-12}$ при n = 22500), що узгоджується з оцінкою r ≲ κ(A)·ε: число обумовленості 5-точкового лапласіана зростає як $O(m^2)$.

4. Час - пам'ять - відносна нев'язка (контраст c = 100)

Середовище вимірювання: Intel Xeon 2,10 ГГц (2 vCPU), 8 ГБ ОЗП, Linux, рушій V8 (Node.js v22). JavaScript виконується в одному потоці. Код не використовує Node-API й запускається в онлайн-компіляторі JS; абсолютні числа на іншій машині будуть іншими, тож для здачі повторіть запуск на своїй машині (висновки про нахили й зони варто перевірити заново).

m	n	LU: с / пам'ять / r	Холецький щільн.: с / пам'ять / r	Стрічк. Холецький: с / пам'ять / r
10	100	0.0005 / 0.08 МБ / 4.4e-15	0.0003 / 0.08 МБ / 5.5e-15	0.0001 / 0.01 МБ / 5.5e-15
15	225	0.0063 / 0.39 МБ / 9.2e-15	0.0026 / 0.39 МБ / 1.2e-14	0.0001 / 0.03 МБ / 1.2e-14
20	400	0.0302 / 1.22 МБ / 2.4e-14	0.0158 / 1.22 МБ / 2.2e-14	0.0002 / 0.06 МБ / 2.2e-14
25	625	0.1306 / 2.98 МБ / 3.3e-14	0.0587 / 2.98 МБ / 3.3e-14	0.0004 / 0.12 МБ / 3.3e-14
30	900	0.3935 / 6.18 МБ / 5.7e-14	0.1842 / 6.18 МБ / 5.9e-14	0.0009 / 0.21 МБ / 5.9e-14
50	2500	—	—	0.0058 / 0.97 МБ / 1.8e-13
100	10000	—	—	0.1082 / 7.71 МБ / 9.9e-13
150	22500	—	—	0.5486 / 25.92 МБ / 2.7e-12

Час - медіана 3-4 вимірів після відкинутого першого запуску. «—» - щільний метод не запускався. Стрічковий Холецький на n = 900 швидший за щільний Холецький приблизно в 200 разів (за LU - в ≈ 440) і використовує приблизно в 30 разів менше пам'яті; на n = 22500 (m = 150) він розв'язує задачу за ≈ 0,5 с, тоді як щільний потребував би ≈ 3,9 ГБ лише на матрицю. Між m = 50 і m = 150 час стрічкового методу зріс у ≈ 95 разів при зростанні m втричі; очікуване $O(m^4)$ дає 81, тобто порядок збігається, а невеликий надлишок можна віднести до ефектів кешу.

5. Щільне подання проти CSR

Пам'ять розрахована аналітично з фактичного nnz: щільне - $8n^2$ байт (double), CSR - 8·nnz (значення) + 4·nnz (індекси стовпців, int32) + 4·(n+1) (початки рядків). CSR будується з трійок (рядок, стовпець, значення) сортуванням підрахунком за O(nnz), а не вставкою елементів по одному.

m	n	nnz	щільне, МБ	CSR, МБ	економія	побудова CSR, мс	A·x щільне, мс	A·x CSR, мс
10	100	460	0.08	0.006	×14	0.006	0.018	0.001

m	n	nnz	щільне, МБ	CSR, МБ	економія	побудова CSR, мс	A·x щільне, мс	A·x CSR, мс
20	400	1920	1.22	0.024	×52	0.014	0.190	0.004
30	900	4380	6.18	0.054	×115	0.026	0.991	0.010
50	2500	12300	47.68	0.150	×317	0.067	7.385	0.027
100	10000	49600	762.94	0.606	×1259	0.273	—	0.109
150	22500	111900	3862.38	1.366	×2827	0.722	—	0.241

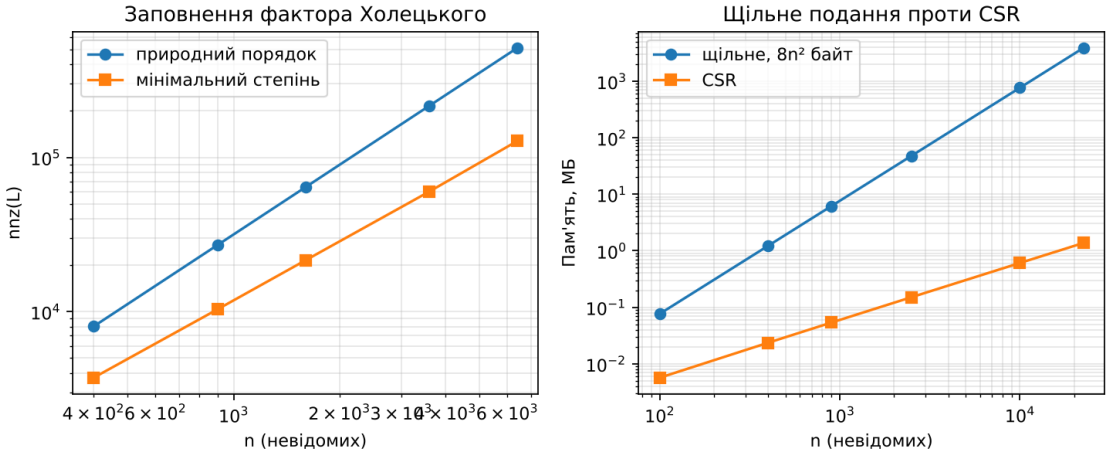


Рис. 2. Ліворуч - заповнення L для двох порядків; праворуч - пам'ять щільного та CSR-подання

Пам'ять щільного подання росте як n^2, CSR - як n; вже при n = 900 економія в ≈ 115 разів, при n = 22500 - у ≈ 2800 разів. Множення A·x у CSR також швидше (в ≈ 20 разів при n = 100 і в ≈ 270 разів при n = 2500), бо не обробляє нулі.

6. Ефект впорядкування: заповнення при факторизації

Заповнення (fill-in) - поява нових ненульових у L там, де в A були нулі. Підраховується символьною факторизацією (гра елімінації на графі) для двох порядків: природного (по рядках сітки) і мінімального степеня (на кожному кроці виключається вершина з найменшим числом сусідів). Правильність символьного підрахунку перевірена чисельно: на сітці m = 12 кількість ненульових у L, отримана щільним Холецьким, збігається з символьною.

```
Перевірка (m=12): природний — символьно 1739, чисельно 1739; мін. степінь — символьно 1026, чисельно 1026
```

m	n	nnz(A, нижн. трикутник)	nnz(L) природний	nnz(L) мін. степінь	заповнення: природн. / мін.	в скільки разів менше
20	400	1160	8019	3729	6859 / 2569	×2.15
30	900	2640	27029	10351	24389 / 7711	×2.61
40	1600	4720	64039	21504	59319 / 16784	×2.98
60	3600	10680	216059	60048	205379 / 49368	×3.60
80	6400	19040	512079	127600	493039 / 108560	×4.01

Конкретні числа: для m = 80 (n = 6400) природний порядок дає 512079 ненульових у L (заповнення 493039), а мінімальний степінь - 127600 (заповнення 108560), тобто в 4,0 разу менше; при m = 20 різниця лише ×2,15. Перевага зростає з n: за даними таблиці nnz(L) природного порядку росте приблизно як n^1,5 (стрічка), а мінімального степеня - приблизно як n^1,3. Це підтверджує п. 4.4 конспекту: порядок рядків і стовпців суттєво впливає на обсяг пам'яті. Факторизація $P \cdot A \cdot P^T$ виконується так само, як A, лише з іншою структурою L.

7. Відповіді на контрольні запитання

№	Відповідь
10	Розклад Холецького вимагає додатної визначеності: на якомусь кроці вираз під коренем стає ≤ 0 (у коді - помилка «не додатно визначена»). Для симетричної, але не додатно визначеної матриці використовують LU з частковим півотингом (або LDL^T з півотингом - спеціалізований варіант для симетричних незнакоозначених матриць).

№	Відповідь
11	Відома «правильна» відповідь x* зазвичай невідома, а похибка в x залежить від обумовленості: навіть ідеальний метод дає похибку x ≈ к(A)·ε. Нев'язка r = ‖Ax - b‖/‖b‖ обчислюється без x*, відразу показує, чи рівняння виконано, і для коректного розв'язувача має бути порядку ε·к(A).
12	Вставка елемента в середину CSR зсуває масиви значень і індексів на O(nnz) елементів, тож n вставок коштують O(nnz²). Правильно: зібрати трійки й побудувати CSR сортуванням підрахунком за O(nnz). У цій роботі побудова для nnz = 111900 зайняла менше 1 мс (див. таблицю в п. 5).
13	Заповнення - нові ненульові в L (чи U) там, де в A були нулі. Виключення вершини з'єднує всіх її залишкових сусідів у кліку; від порядку виключення залежить розмір цих кліків. Для сітки m = 80 різниця між природним порядком і мінімальним степенем - у 4,0 разу за nnz(L) (див. п. 6).
14	Явне обчислення оберненої матриці виправдане лише тоді, коли потрібна сама обернена матриця (її елементи) як результат, наприклад коваріаційна матриця або функція Гріна. Для розв'язання Ax = b обернену не обчислюють: факторизація дешевша й точніша. Сформульовано за загальноприйнятою практикою; точне формулювання «єдиного випадку» з конспекту лекції варто звірити.
15	Асимптотика однакова, O(n³), але різна константа: наївний код погано використовує кеш (доступ до B по стовпцях), не використовує SIMD і багатоядерність, а бібліотека (BLAS) використовує блокування під кеш, векторні інструкції й паралелізм. У нашому JS-експерименті (лаба 4) оптимізоване ядро було швидшим за наївну лише у ≈ 3 рази, бо SIMD і потоки в чистому JS недоступні.
16	Запуск: вставити lab6.js в онлайн-компілятор JavaScript і натиснути Run (≈ 10 с). Якщо компілятор обриває виконання, у циклах секцій 2 і 3 вилучіть найбільші m (100, 150).

Додаток. Повний код lab6.js

```
const now = (typeof performance !== "undefined" && performance.now)
  ? () => performance.now() : () => Date.now();
const median = (a) => { const s = a.slice().sort((x, y) => x - y), m = s.length >> 1;
  return s.length % 2 ? s[m] : (s[m - 1] + s[m]) / 2; };
function timeMedian(fn, repeats = 4) {
  const t = [];
  for (let r = 0; r < repeats; r++) { const t0 = now(); fn(); if (r > 0) t.push((now() - t0) / 1000); }
  return median(t);
}
const MB = (bytes) => (bytes / 1048576).toFixed(2) + " МБ";

function buildGrid(m, contrast) {
  const n = m * m;
  const kOf = (r, c) => (c < m / 2 ? 1 : contrast);
  const harm = (a, b) => 2 * a * b / (a + b);
  const rows = [], cols = [], vals = [];
  const diag = new Float64Array(n);
  for (let r = 0; r < m; r++) for (let c = 0; c < m; c++) {
    const i = r * m + c, k = kOf(r, c);
    for (const [dr, dc] of [[-1, 0], [1, 0], [0, -1], [0, 1]]) {
      const rr = r + dr, cc = c + dc;
      if (rr < 0 || rr >= m || cc < 0 || cc >= m) { diag[i] += k; continue; }
      const w = harm(k, kOf(rr, cc));
      diag[i] += w;
      rows.push(i); cols.push(rr * m + cc); vals.push(-w);
    }
  }
  for (let i = 0; i < n; i++) { rows.push(i); cols.push(i); vals.push(diag[i]); }
  const b = new Float64Array(n).fill(1);
  return { n, m, rows, cols, vals, b };
}

function toCSR(n, rows, cols, vals) {
  const rowPtr = new Int32Array(n + 1);
  for (const r of rows) rowPtr[r + 1]++;
  for (let i = 0; i < n; i++) rowPtr[i + 1] += rowPtr[i];
  const nnz = rows.length, colIdx = new Int32Array(nnz), val = new Float64Array(nnz);
  const next = rowPtr.slice(0, n);
  for (let t = 0; t < nnz; t++) { const p = next[rows[t]]++; colIdx[p] = cols[t]; val[p] = vals[t]; }
  return { n, nnz, rowPtr, colIdx, val };
}
function csrMul(A, x, y) {
  for (let i = 0; i < A.n; i++) {
    let s = 0;
    for (let p = A.rowPtr[i]; p < A.rowPtr[i + 1]; p++) s += A.val[p] * x[A.colIdx[p]];
    y[i] = s;
  }
}
function toDense(A) {
  const D = new Float64Array(A.n * A.n);
  for (let i = 0; i < A.n; i++) for (let p = A.rowPtr[i]; p < A.rowPtr[i + 1]; p++) D[i * A.n + A.colIdx[p]] =
    A.val[p];
  return D;
}
function denseMul(D, n, x, y) {
  for (let i = 0; i < n; i++) { let s = 0; const o = i * n; for (let j = 0; j < n; j++) s += D[o + j] * x[j];
    y[i] = s; }
}
const norm = (v) => Math.sqrt(v.reduce((s, x) => s + x * x, 0));
function relResidual(A, x, b) {
  const y = new Float64Array(A.n); csrMul(A, x, y);
  for (let i = 0; i < A.n; i++) y[i] -= b[i];
  return norm(y) / norm(b);
}

function solveLU(Ain, n, b) {
  const A = Float64Array.from(Ain), x = Float64Array.from(b);
  for (let k = 0; k < n; k++) {
    let p = k, mx = Math.abs(A[k * n + k]);
    for (let i = k + 1; i < n; i++) { const v = Math.abs(A[i * n + k]); if (v > mx) { mx = v; p = i; } }

    if (mx === 0) throw new Error("Матриця вироджена");
    if (p !== k) {
      for (let j = 0; j < n; j++) { const t = A[k * n + j]; A[k * n + j] = A[p * n + j]; A[p * n + j] = t; }
      const t = x[k]; x[k] = x[p]; x[p] = t;
    }
    const d = A[k * n + k];
    for (let i = k + 1; i < n; i++) {
      const f = A[i * n + k] / d;
      A[i * n + k] = f;
      for (let j = k + 1; j < n; j++) A[i * n + j] -= f * A[k * n + j];
      x[i] -= f * x[k];
    }
  }
  for (let i = n - 1; i >= 0; i--) {
    let s = x[i];
    for (let j = i + 1; j < n; j++) s -= A[i * n + j] * x[j];
    x[i] = s / A[i * n + i];
```

```
  }
  return x;
}

function choleskyDense(Ain, n) {
  const L = new Float64Array(n * n);
  for (let j = 0; j < n; j++) {
    let d = Ain[j * n + j];
    for (let k = 0; k < j; k++) d -= L[j * n + k] * L[j * n + k];
    if (d <= 0) throw new Error("Матриця не додатно визначена");
    const ljj = Math.sqrt(d); L[j * n + j] = ljj;
    for (let i = j + 1; i < n; i++) {
      let s = Ain[i * n + j];
      for (let k = 0; k < j; k++) s -= L[i * n + k] * L[j * n + k];
      L[i * n + j] = s / ljj;
    }
  }
  return L;
}
function solveCholeskyDense(Ain, n, b) {
  const L = choleskyDense(Ain, n), y = Float64Array.from(b);
  for (let i = 0; i < n; i++) { let s = y[i]; for (let k = 0; k < i; k++) s -= L[i * n + k] * y[k]; y[i] = s /
    L[i * n + i]; }
  for (let i = n - 1; i >= 0; i--) { let s = y[i]; for (let k = i + 1; k < n; k++) s -= L[k * n + i] * y[k];
    y[i] = s / L[i * n + i]; }
  return y;
}

function solveBandCholesky(A, w, b) {
  const n = A.n, W = w + 1, L = new Float64Array(n * W);
  for (let i = 0; i < n; i++) for (let p = A.rowPtr[i]; p < A.rowPtr[i + 1]; p++) {
    const j = A.colIdx[p]; if (j <= i && i - j <= w) L[i * W + (i - j)] = A.val[p];
  }
  for (let i = 0; i < n; i++) {
    const j0 = Math.max(0, i - w);
    for (let j = j0; j <= i; j++) {
      let s = L[i * W + (i - j)];
      const k0 = Math.max(j0, j - w);
      for (let k = k0; k < j; k++) s -= L[i * W + (i - k)] * L[j * W + (j - k)];
      if (j === i) { if (s <= 0) throw new Error("не SPD"); L[i * W] = Math.sqrt(s); }
      else L[i * W + (i - j)] = s / L[j * W];
    }
  }
  const y = Float64Array.from(b);
  for (let i = 0; i < n; i++) { let s = y[i]; for (let k = Math.max(0, i - w); k < i; k++) s -= L[i * W + (i -
    k)] * y[k]; y[i] = s / L[i * W]; }
  for (let i = n - 1; i >= 0; i--) {
    let s = y[i];
    for (let k = i + 1; k <= Math.min(n - 1, i + w); k++) s -= L[k * W + (k - i)] * y[k];
    y[i] = s / L[i * W];
  }
  return y;
}

function adjacencyOf(A) {
  const adj = Array.from({ length: A.n }, () => new Set());
  for (let i = 0; i < A.n; i++) for (let p = A.rowPtr[i]; p < A.rowPtr[i + 1]; p++) {
    const j = A.colIdx[p]; if (j !== i) adj[i].add(j);
  }
  return adj;
}
function symbolicNnzL(adjIn, minDegree) {
  const n = adjIn.length, adj = adjIn.map(s => new Set(s)), done = new Uint8Array(n), perm = [];
  let nnz = 0;
  for (let step = 0; step < n; step++) {
    let v = step;
    if (minDegree) { let best = Infinity; for (let u = 0; u < n; u++) if (!done[u] && adj[u].size < best) {
    best = adj[u].size; v = u; } }
    done[v] = 1; perm.push(v);
    const nb = Array.from(adj[v]);
    nnz += nb.length + 1;
    for (const a of nb) {
      adj[a].delete(v);
      for (const c of nb) if (c !== a) adj[a].add(c);
    }
    adj[v].clear();
  }
  return { nnz, perm };
}
function permuteDense(D, n, perm) {
  const P = new Float64Array(n * n);
  for (let i = 0; i < n; i++) for (let j = 0; j < n; j++) P[i * n + j] = D[perm[i] * n + perm[j]];
  return P;
}
function countNnzLower(L, n) { let c = 0; for (let i = 0; i < n; i++) for (let j = 0; j <= i; j++) if
    (Math.abs(L[i * n + j]) > 1e-14) c++; return c; }

console.log("=== 1. Коректність: нев'язка r = ||Ax-b||/||b||, m = 20, n = 400 ===");
{
  const m = 20;
  console.log("контраст | LU (півотинг)  | Холецький (щільний) | Холецький (стрічковий)");
```

```
  for (const c of [1, 10, 100, 1000]) {
    const g = buildGrid(m, c), A = toCSR(g.n, g.rows, g.cols, g.vals), D = toDense(A);
    const r1 = relResidual(A, solveLU(D, g.n, g.b), g.b);
    const r2 = relResidual(A, solveCholeskyDense(D, g.n, g.b), g.b);
    const r3 = relResidual(A, solveBandCholesky(A, m, g.b), g.b);
    console.log(`${String(c).padEnd(9)}| ${r1.toExponential(2).padEnd(15)}| ${r2.toExponential(2).padEnd(20)}|
    ${r3.toExponential(2)}`);
  }
}

console.log("\n=== 2. Час — пам'ять — нев'язка (контраст = 100) ===");
console.log("m   n      | LU: с / пам'ять / r           | Холецький щільн.: с / пам'ять / r | Стрічк.
    Холецький: с / пам'ять / r");
for (const m of [10, 15, 20, 25, 30, 50, 100, 150]) {
  const g = buildGrid(m, 100), A = toCSR(g.n, g.rows, g.cols, g.vals), n = g.n;
  let s1 = "—", s2 = "—";
  if (m <= 30) {
    const D = toDense(A);
    const tl = timeMedian(() => solveLU(D, n, g.b)), rl = relResidual(A, solveLU(D, n, g.b), g.b);
    const tc = timeMedian(() => solveCholeskyDense(D, n, g.b)), rc = relResidual(A, solveCholeskyDense(D, n,
    g.b), g.b);
    s1 = `${tl.toFixed(4)} / ${MB(8 * n * n)} / ${rl.toExponential(1)}`;
    s2 = `${tc.toFixed(4)} / ${MB(8 * n * n)} / ${rc.toExponential(1)}`;
  }
  const tb = timeMedian(() => solveBandCholesky(A, m, g.b), 3), rb = relResidual(A, solveBandCholesky(A, m,
    g.b), g.b);
  console.log(`${String(m).padEnd(4)}${String(n).padEnd(7)}| ${s1.padEnd(29)}| ${s2.padEnd(34)}|
    ${tb.toFixed(4)} / ${MB(8 * n * (m + 1))} / ${rb.toExponential(1)}`);
}

console.log("\n=== 3. Щільне подання проти CSR (контраст = 100) ===");
console.log("m   n      nnz     | щільне, МБ | CSR, МБ | економія | побудова CSR, мс | A·x щільне, мс | A·x

    CSR, мс");
for (const m of [10, 20, 30, 50, 100, 150]) {
  const g = buildGrid(m, 100), n = g.n;
  const tBuild = timeMedian(() => toCSR(n, g.rows, g.cols, g.vals), 5) * 1000;
  const A = toCSR(n, g.rows, g.cols, g.vals);
  const denseBytes = 8 * n * n, csrBytes = 8 * A.nnz + 4 * A.nnz + 4 * (n + 1);
  const x = new Float64Array(n).fill(1), y = new Float64Array(n);
  const reps = 50;
  const tCsr = timeMedian(() => { for (let r = 0; r < reps; r++) csrMul(A, x, y); }, 5) * 1000 / reps;
  let tDense = "—";
  if (m <= 50) { const D = toDense(A); tDense = (timeMedian(() => { for (let r = 0; r < 5; r++) denseMul(D, n,
    x, y); }, 5) * 1000 / 5).toFixed(3); }
  console.log(`${String(m).padEnd(4)}${String(n).padEnd(7)}${String(A.nnz).padEnd(8)}| ${(denseBytes /
    1048576).toFixed(2).padEnd(11)}| ` +
    `${(csrBytes / 1048576).toFixed(3).padEnd(8)}| ×${(denseBytes / csrBytes).toFixed(0).padEnd(8)}|
    ${tBuild.toFixed(3).padEnd(17)}| ${String(tDense).padEnd(15)}| ${tCsr.toFixed(3)}`);
}

console.log("\n=== 4. Ефект впорядкування: заповнення L ===");
{
  const g = buildGrid(12, 1), A = toCSR(g.n, g.rows, g.cols, g.vals), D = toDense(A), adj = adjacencyOf(A);
  const nat = symbolicNnzL(adj, false), md = symbolicNnzL(adj, true);
  const numNat = countNnzLower(choleskyDense(D, g.n), g.n);
  const numMd = countNnzLower(choleskyDense(permuteDense(D, g.n, md.perm), g.n), g.n);
  console.log(`Перевірка (m=12): природний — символьно ${nat.nnz}, чисельно ${numNat}; мін. степінь — символьно
    ${md.nnz}, чисельно ${numMd}`);
}
console.log("m   n      nnz(A,нижн.) | nnz(L) природн. | nnz(L) мін.степ. | заповнення природн. / мін.степ. | у
    скільки разів менше");
for (const m of [20, 30, 40, 60, 80]) {
  const g = buildGrid(m, 1), A = toCSR(g.n, g.rows, g.cols, g.vals), n = g.n, adj = adjacencyOf(A);
  let lowerA = 0; for (let i = 0; i < n; i++) for (let p = A.rowPtr[i]; p < A.rowPtr[i + 1]; p++) if
    (A.colIdx[p] <= i) lowerA++;
  const nat = symbolicNnzL(adj, false), md = symbolicNnzL(adj, true);
  console.log(`${String(m).padEnd(4)}${String(n).padEnd(7)}${String(lowerA).padEnd(13)}|
    ${String(nat.nnz).padEnd(16)}| ${String(md.nnz).padEnd(17)}| ` +
    `${(nat.nnz - lowerA) + " / " + (md.nnz - lowerA)} | ×${(nat.nnz / md.nnz).toFixed(2)}`);
}
```